Aula 05 - Prof. Vinícius Borges e Raphael Lacerda

SEFAZ-AL (Auditor Fiscal da Administração Tributária Estadual - AFTE) Desenvolvimento de Sistemas - 2026 (Pós-Edital)

Autor:
Vinicius Borges

27 de Agosto de 2026

02648216103 - HENRIQUE AMORIM BARRETO

Índice

1) DevOps - Teoria 3

2) DevOps - Questões Comentadas 20

3) DevOps - Lista de Questões 36

02648216103 - HENRIQUE AMORIM BARRETO

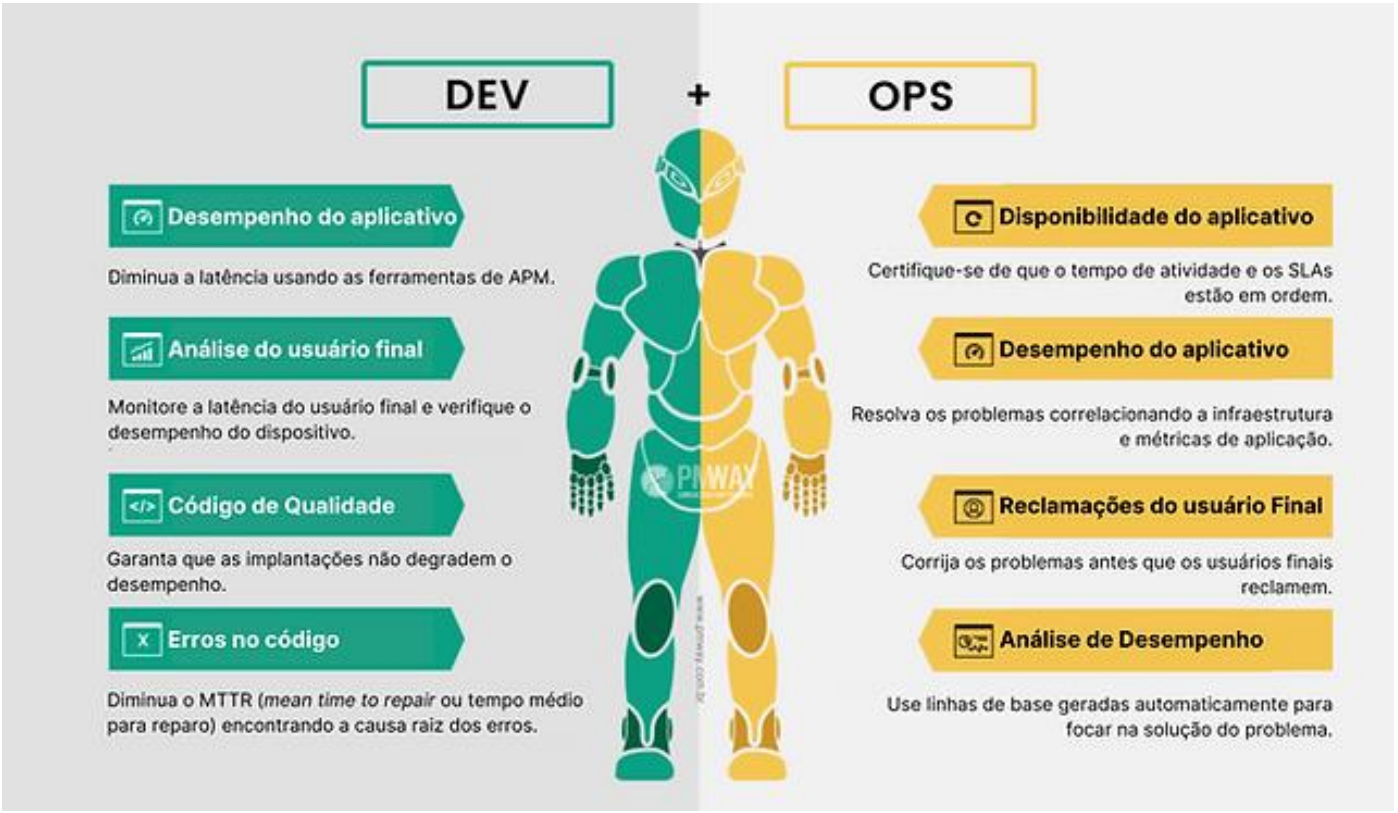
DEVOPS

Conceitos Gerais

Nos primórdios dos desenvolvimentos de sistemas, o pessoal da TI era dividido em dois grupos: os desenvolvedores, responsáveis por criar o programa (da perspectiva dos códigos), e os especialistas em infraestrutura, responsáveis por implementar os programas e mantê-los em funcionamento.

Um grupo não costumava se dava bem com o outro, e isso resultava em um desenvolvimento cheio de problemas, dificuldades de implementação de mudanças, uma implementação cheia de defeitos e erros.

Para resolver isso, uma integração entre desenvolvedores e operadores foi desenvolvida – resultando em **DevOps**. DevOps é uma **junção entre "Development" e "Operation"**, de forma a alinhar o trabalho dos desenvolvedores com o ambiente de implementação, buscando aumentar a **eficiência**, **velocidade** e **segurança** do desenvolvimento de aplicações.



O termo chave do DevOps é a **colaboração**, visando fazer **entregas contínuas**, e realizar **melhorias contínuas** através de um **feedback constante**. É comum que você veja o DevOps associado a um símbolo do infinito, já que tudo no DevOps funciona em um ciclo constante, sem interrupções.

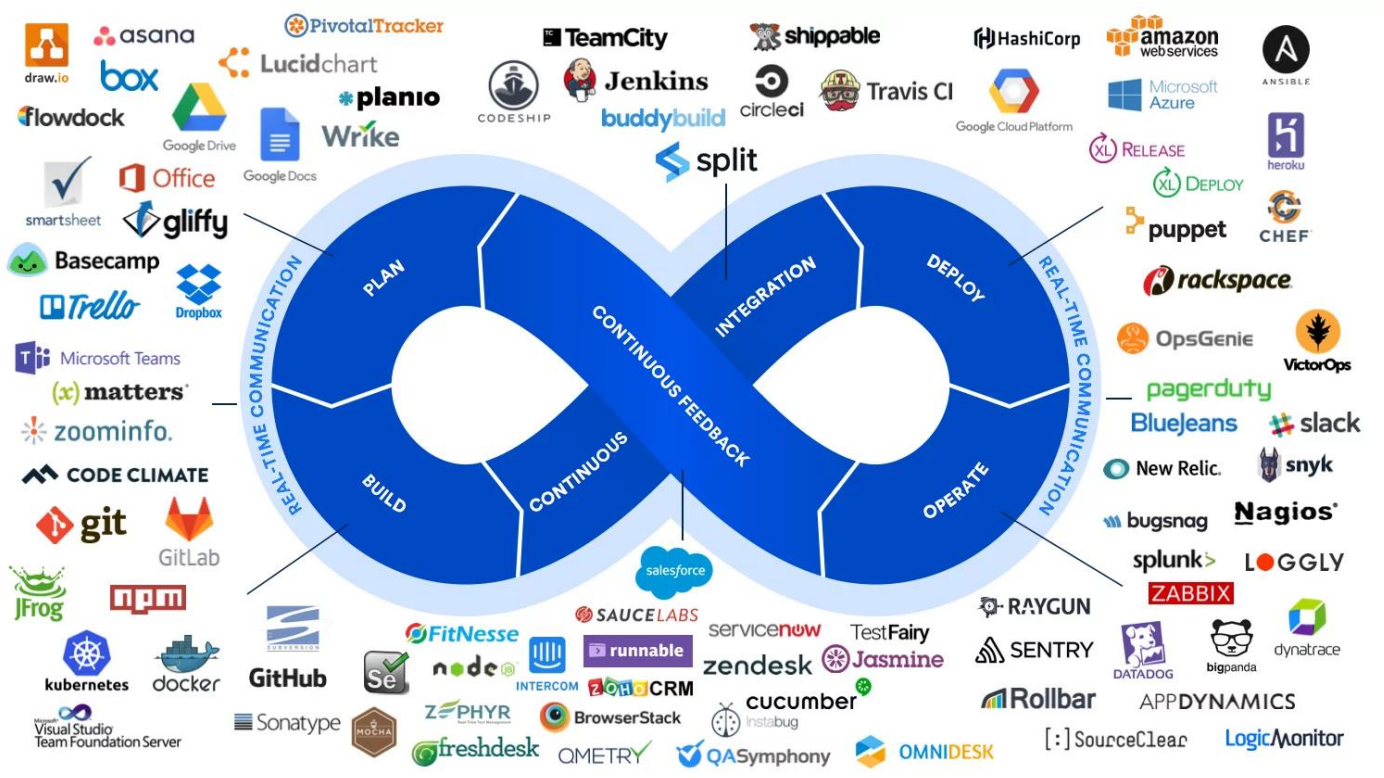
Além disso, o DevOps tem um grande foco na **automação do processo**. Essa automação permite uma entrega mais constante, com menos erros e mais eficiente.
Sendo assim, podemos definir o DevOps como um conjunto de práticas, com foco na aceleração, colaboração, *feedback* e melhoria iterativa para o desenvolvimento de sistemas e operações. Usualmente,

vemos um foco do DevOps na equipe, implementando metodologias Ágeis e Lean, e uma equipe multifuncional.

A equipe multifuncional é um ponto muito importante, já que um dos objetivos do DevOps é **reduzir os silos organizacionais**. Silos organizacionais, ou empresariais, ocorrem quando setores distintos trabalham de forma independente, sem integração, sem comunicação, agindo somente conforme suas vontades internas.

O ambiente DevOps compreende diversas plataformas:

- Virtualização
- Contêiners
- Automação de configuração
- Automação de implementações
- Versionamento
- Entrega contínua



Cada uma dessas plataformas contribui para uma parte diferente do ciclo DevOps. Temos, por exemplo, Kubernetes, Docker, Jenkins, Grafana, Ansible, Puppet, entre vários outros que veremos ao longo desse módulo. Como você pode ter percebido na imagem acima, temos uma infinidade de programas e plataformas de DevOps.

Princípios

Para implementar seus objetivos, o DevOps conta com **6 princípios**. Como não temos uma bibliografia oficial, temos várias linhas – 4 princípios, 5 princípios, entre outras. Vou pela mais extensa, que acaba abrangendo todos os principais princípios.

Colaboração

A colaboração é a espinha dorsal do DevOps. Envolve a quebra das barreiras tradicionais entre as equipes de desenvolvimento e operações, promovendo a comunicação aberta e a colaboração constante. Isso ajuda a evitar silos de informações e a melhorar a eficiência ao longo do ciclo de vida do desenvolvimento de software.

Decisão baseada em dados

Tomar decisões com base em dados é crucial para o sucesso do DevOps. A coleta e análise de dados permitem uma compreensão mais profunda do desempenho do software, dos processos e das interações entre as equipes. Isso leva a decisões mais assertivas, melhorando a qualidade do software e a eficiência operacional.

Decisão centrada no cliente

Colocar o cliente no centro das decisões é fundamental para garantir que o software atenda às necessidades reais e às expectativas do usuário final. O feedback do cliente é valorizado, e os processos de desenvolvimento são orientados para fornecer continuamente valor tangível aos usuários.

Melhoria Contínua e Monitoramento

A busca constante por melhorias é um dos pilares do DevOps. Isso envolve avaliar regularmente os processos, identificar áreas de oportunidade e implementar mudanças incrementais para aprimorar a eficiência, a qualidade e a satisfação do cliente ao longo do tempo. As informações para melhoria devem advir de um fluxo de *feedback* contínuo.

Atribuição de responsabilidade

Em um ambiente DevOps, a responsabilidade é distribuída entre as equipes de desenvolvimento e operações. Isso significa que todos compartilham a responsabilidade pela entrega e operação do software. Evita-se a mentalidade de "não é meu trabalho" e promove um senso de propriedade coletiva.

Automação

A automação é fundamental para acelerar os processos de desenvolvimento, teste e implementação. Ao automatizar tarefas repetitivas e propensas a erros, as equipes conseguem liberar atualizações mais rapidamente e com maior confiabilidade. Isso também permite a consistência no ambiente de produção, reduzindo o risco de falhas devido a erros humanos.

02648216103 - HENRIQUE AMORIM BARRETO

Fases da Implementação do DevOps

Desde o nascimento do DevOps, tivemos diversas fases, movidas pelas diferentes formas de arquitetura. Por exemplo, antigamente o desenvolvimento de softwares era quase sempre monolítico; agora, a perspectiva passa a ser muito maior para microserviços e SOA. E com isso, o DevOps vai acompanhando as mudanças, amadurecendo e mudando junto.

Temos 4 fases para o DevOps – segundo o GitLab:

TRAGA SEU PRÓPRIO DEVOPS

Na fase Traga seu Próprio DevOps, cada equipe selecionava suas próprias ferramentas. Essa abordagem causava problemas quando as equipes tentavam trabalhar juntas, pois não estavam familiarizadas com as ferramentas das outras equipes.

DEVOPS DE MELHOR QUALIDADE

Para lidar com os desafios de usar ferramentas díspares, as organizações avançaram para a segunda fase, DevOps de Melhor Qualidade. Nesta fase, as organizações padronizaram no mesmo conjunto de ferramentas, com uma ferramenta preferida para cada etapa do ciclo de vida do DevOps. Isso ajudou as equipes a colaborarem entre si, mas o problema tornou-se mover as mudanças de software através das ferramentas para cada etapa.

DIY DEVOPS

Para remediar esse problema, as organizações adotaram o DevOps DIY (DIY vem do inglês Do It Yourself, ou Faça Você Mesmo), construindo sobre e entre suas ferramentas. Realizaram muito trabalho personalizado para integrar suas soluções pontuais de DevOps. No entanto, como essas ferramentas foram desenvolvidas independentemente, sem integração em mente, nunca se encaixavam perfeitamente. Para muitas organizações, manter o DIY DevOps era um esforço significativo e resultava em custos mais altos, com engenheiros mantendo a integração de ferramentas em vez de trabalhar no produto de software principal.

PLATAFORMA DEVOPS

Uma abordagem de plataforma única melhora a experiência da equipe e a eficiência nos negócios. Uma plataforma DevOps substitui o DIY DevOps, permitindo visibilidade e controle em todas as etapas do ciclo de vida do DevOps.

Ao capacitar todas as equipes - Desenvolvimento, Operações, TI, Segurança e Negócios - a planejar, construir, proteger e implantar software colaborativamente em um sistema unificado de ponta a ponta, uma plataforma DevOps representa uma mudança fundamental para realizar todo o potencial do DevOps.

Usualmente, como plataforma de DevOps, usa-se **plataformas FaaS**, ou **Function as a Service** (função como serviço), uma forma de serviço de computação em nuvem, que permite a execução de códigos em resposta a eventos, sem necessidade de gerenciar uma infraestrutura complexa. As FaaS são um tipo de forma de desenvolvimento *serverless*, permitindo colaboração, agilidade, escalabilidade e automação. Veremos mais sobre as FaaS à frente.

Infraestrutura como Código (IaC)

Infraestrutura como Código, ou o inglês ***Infrastructure as Code (IaC)***, refere-se à prática de gerenciar e provisionar a infraestrutura de TI por meio de descrições e scripts codificados, ao invés de configurações manuais. Esses scripts são tratados como código-fonte e podem ser versionados, testados, revisados e integrados aos sistemas de controle de origem.
A adoção de IaC traz diversos benefícios, incluindo:

- **Reprodutibilidade:** Ambientes podem ser replicados exatamente da mesma maneira, eliminando inconsistências.
- **Escalabilidade:** A infraestrutura pode ser dimensionada automaticamente conforme a demanda.
- **Versionamento:** Todas as alterações são rastreáveis, permitindo rollbacks ou auditorias.
- **Colaboração:** Equipes de desenvolvimento e operações podem colaborar efetivamente na criação e manutenção da infraestrutura.
- **Automatização:** A automação do provisionamento e da configuração simplifica e acelera os processos.

O código de automação pode assumir dois formatos – usando uma **linguagem declarativa**, e usando uma **linguagem imperativa**. A **linguagem imperativa** necessita que especifiquemos explicitamente os passos necessários para configurar a infraestrutura. Aqui, o **foco está no "como**" configurar os recursos, detalhando as etapas e comandos que a ferramenta deve executar.

Já na abordagem **declarativa**, **descrevemos o estado final desejado** da infraestrutura, e a ferramenta de IaC é responsável por realizar as ações necessárias para alcançar esse estado. O foco está no "o quê" queremos configurar, e não em "como" essa configuração será realizada. O usuário especifica o resultado esperado (por exemplo, "desejo 3 instâncias EC2 em execução na AWS") e a ferramenta de IaC identifica as mudanças necessárias para transformar o ambiente atual no estado desejado.

Aspecto	Declarativa	Imperativa
Foco	Estado final desejado	Sequência de comandos e ações
Idempotência	Sim	Não
Complexidade	Menor, mais intuitiva	Maior, exige detalhamento
Controle	Automatizado pela ferramenta	Totalmente manual
Exemplo de Ferramentas	Terraform, Ansible (modo declarativo)	AWS CLI, scripts Bash ou Python

Uma das formas mais comuns para definirmos infraestrutura é a partir de **arquivos YAML**, Yet Another Markup Language, é uma linguagem legível para seres humanos de **serialização de dados,** usada na escrita de configurações. Ela usa espaços em branco para estruturas os dados, criando uma hierarquia pela endentação (recuos em relação à margem esquerda), e funcionando em pares chave-valor. Veja uma parte de um arquivo YAML:

02648216103 - HENRIQUE AMORIM BARRETO

YAML

```yaml
- name: Configuração de Servidor Web
  hosts: servidores_web
  become: true
  tasks:
    - name: Instalar o servidor web Apache
      apt:
        name: apache2
        state: present
    - name: Habilitar o serviço Apache
      systemd:
        name: apache2
        enabled: yes
        state: started
```

(CESPE/ANALISTA FAZENDÁRIO PREF. FORTALEZA/2023) A equipe de redes de um órgão público está trabalhando para auxiliar no cumprimento das metas da equipe de desenvolvimento de sistemas do mesmo órgão e vislumbrou a possibilidade de utilização de DevOps. Para tal, a equipe de redes indicou a contratação de uma API em uma nuvem. A API indicada permite que os desenvolvedores e os administradores dos sistemas interajam com a infraestrutura de modo programático e em escala, evitando a instalação e a configuração dos recursos manualmente todas as vezes que precisam recriar um ambiente de desenvolvimento.

Para essa atividade, a equipe de desenvolvimento utilizou a prática DevOps de:

a) comunicação e colaboração;
b) integração contínua;
c) entrega contínua;
d) microsserviços;
e) infraestrutura como código.

Comentários:

A chave da questão está aqui:

"*permite que os desenvolvedores e os administradores dos sistemas interajam com a infraestrutura de modo programático e em escala, evitando a instalação e a configuração dos recursos manualmente*"

A prática que permite implementar essa necessidade, fazendo com que não haja a necessidade de configuração manual em cada ambiente de trabalho que o aplicativo for implementado, é a *infraestrutura como código*. (*Gabarito: Letra E)*

Conceitos Fundamentais do DevOps

Temos 5 conceitos fundamentais para fazer o DevOps funcionar. Dois deles falaremos mais à frente, que é a **Integração Contínua** e a **Entrega Contínua**, que dão origem à **pipeline CI/CD** (Continuous Integration/Continuous Delivery).

Os outros conceitos envolvem:

- Emprego de **controle de versionamento**, para auditoria do código. No curso, veremos a ferramenta GIT para versionamento.
- Emprego de **metodologias ágeis**, como Scrum, XP e KanBan.
- **Testes à esquerda**



Controle de Versionamento

O **controle de versionamento** é um sistema que **registra alterações** feitas em arquivos ou conjuntos de arquivos ao longo do tempo, **permitindo que versões anteriores sejam recuperadas** ou que mudanças específicas sejam analisadas. Ele é amplamente utilizado no desenvolvimento de software, mas também é aplicável a documentos, configurações de infraestrutura e outros tipos de arquivos.

O objetivo do controle de versionamento é **gerenciar e acompanhar alterações no tempo**, garantindo organização, rastreabilidade, colaboração eficiente entre equipes e segurança para reverter mudanças em caso de erro. A principal ferramenta para versionamento é o Git, que, se constar no seu edital, será aprofundada em momento oportuno.

Metodologias Ágeis

As **metodologias ágeis** são **abordagens de gestão e desenvolvimento de projetos** que enfatizam a colaboração, a flexibilidade, a entrega contínua de valor e a adaptação às mudanças. Originadas do Manifesto Ágil de 2001, essas metodologias **priorizam as pessoas e suas interações em detrimento de processos rígidos**, focam na entrega de software funcional ao invés de documentações extensas, e valorizam a capacidade de responder a mudanças mais do que seguir planos predefinidos. Elas são amplamente utilizadas em desenvolvimento de software, mas seus princípios também podem ser aplicados em outras áreas de projetos complexos e dinâmicos.

Entre as principais metodologias ágeis estão o **Scrum**, que organiza o trabalho em sprints para alcançar entregas frequentes, e o **Kanban**, que utiliza fluxos visuais para gerenciar tarefas. Outras abordagens, como **Extreme Programming (XP)** e **Lean Development**, também compartilham os valores ágeis. O objetivo comum dessas metodologias é entregar valor de forma incremental e contínua, envolvendo o cliente no processo e promovendo ciclos curtos de feedback, o que garante maior alinhamento com os requisitos e necessidades reais do projeto. Essa abordagem ágil contrasta com métodos tradicionais, como o modelo cascata, que seguem um planejamento linear e menos flexível.

Testes à Esquerda

O **testes à esquerda** (ou *shift-left testing*) é uma abordagem no desenvolvimento de software que envolve a **antecipação e a integração da prática de testes**, desde as fases iniciais do ciclo de vida do desenvolvimento, A ideia é antecipar a detecção de defeitos e evitar o *deploy* de defeitos em serviços ao vivo.

Um cuidado é que os testes à esquerda diferem um pouco do TDD – Test Driven Development. No TDD, nós projetamos os testes antes e, com base neles, desenvolvemos o software. Na prática, esse processo é muito engessado e não permite maleabilidade. Os testes à esquerda são uma flexibilização do TDD.

Alguns princípios do *shift-left testing* incluem:

- Teste antecipadamente
- Faça testes colaborativos, multifuncionais.
- Automatize testes
- Aplique testes unitários contínuos
- Faça uma integração contínua
- Revise o código com foco no resultado dos testes
- Preze pela qualidade como cultura

(CESPE/SEPLAN RR/2023) No que diz respeito aos conceitos de criptografia, à assinatura digital, aos conceitos utilizados em sistemas operacionais e às noções de DevOps, julgue o item seguinte.

Colaboração multidisciplinar, teste manual no final do desenvolvimento e implantação contínua são algumas das características marcantes da cultura DevOps.

Comentários:

Está quase tudo certo – mas temos um erro gritante. Tudo que envolve DevOps busca ocorrer de forma **automatizada**, então a afirmativa incorre em erro ao dizer que temos **testes manuais**, temos sim testes, mas esses ocorrem de forma automatizada. *(Gabarito: Errado)*

Colaboração entre equipes

Um dos pilares mais importantes do DevOps é justamente a colaboração entre equipes que, tradicionalmente, operavam de forma isolada. No modelo clássico de desenvolvimento de software, tínhamos três grandes "silos": a equipe de Desenvolvimento (Dev), responsável por escrever o código; a equipe de Quality Assurance (QA), responsável por testar e garantir a qualidade do software; e a equipe de

Operações (Ops), responsável por implantar e manter os sistemas em produção. Cada grupo tinha seus próprios objetivos, ferramentas e métricas de sucesso, o que frequentemente gerava conflitos.

Pense assim: o Dev quer entregar rápido, o Ops quer estabilidade, e o QA quer qualidade. Sem integração, esses objetivos colidem. O desenvolvedor empurra código sem testar direito, o QA descobre problemas tarde demais, e o Ops recebe um software instável para colocar em produção. O DevOps surge exatamente para quebrar esse ciclo, promovendo responsabilidade compartilhada e comunicação contínua entre todos os envolvidos.

No ambiente DevOps, essas três equipes passam a trabalhar de forma integrada, compartilhando ferramentas, processos e, principalmente, responsabilidade pelo resultado final. O Dev não "joga o código por cima do muro" para o QA testar; o QA não é apenas um "portão" que barra entregas; e o Ops não fica esperando passivamente para receber algo pronto. Todos participam do ciclo completo, desde o planejamento até o monitoramento em produção.

Essa integração se materializa em práticas concretas: **reuniões conjuntas** de planejamento (onde Dev, QA e Ops discutem juntos os requisitos e riscos), **pair programming** entre desenvolvedores e testadores, revisão de código cruzada entre equipes, e **post-mortems** colaborativos quando incidentes ocorrem em produção. A ideia central é que a qualidade e a estabilidade não são responsabilidade de uma equipe específica, mas sim de todos.

O Papel do QA em Projetos DevOps

No modelo tradicional (cascata), o QA atuava apenas no final do ciclo de desenvolvimento, como um "portão" que barrava ou liberava o software para produção. Isso gerava um problema sério: os defeitos eram descobertos tarde, quando o custo de correção já era altíssimo. No DevOps, o QA assume um papel completamente diferente, atuando de forma contínua e integrada ao longo de todo o pipeline.

Vamos entender as principais atribuições do QA no contexto DevOps:

- **Automação de testes**: O QA cria e mantém suítes de testes automatizados são executados automaticamente na pipeline CI/CD. Isso é o coração da garantia de qualidade contínua.
- **Shift-left testing**: Conforme já vimos na seção de Testes à Esquerda, o QA antecipa a validação para as fases iniciais do desenvolvimento, participando desde o planejamento e a definição dos requisitos.
- **Definição de critérios de qualidade**: O QA ajuda a definir padrões de aceitação, métricas de qualidade e gates (portões) de qualidade que devem ser cumpridos antes que o código avance nas etapas da pipeline.
- **Monitoramento pós-deploy**: O QA colabora com Ops no monitoramento da aplicação em produção, analisando logs, métricas de performance e taxas de erro para identificar regressões rapidamente.
- **Cultura de qualidade**: Mais do que testar, o QA no DevOps atua como evangelizador da qualidade, disseminando boas práticas de teste entre os desenvolvedores e promovendo a mentalidade de que qualidade é responsabilidade de todos.

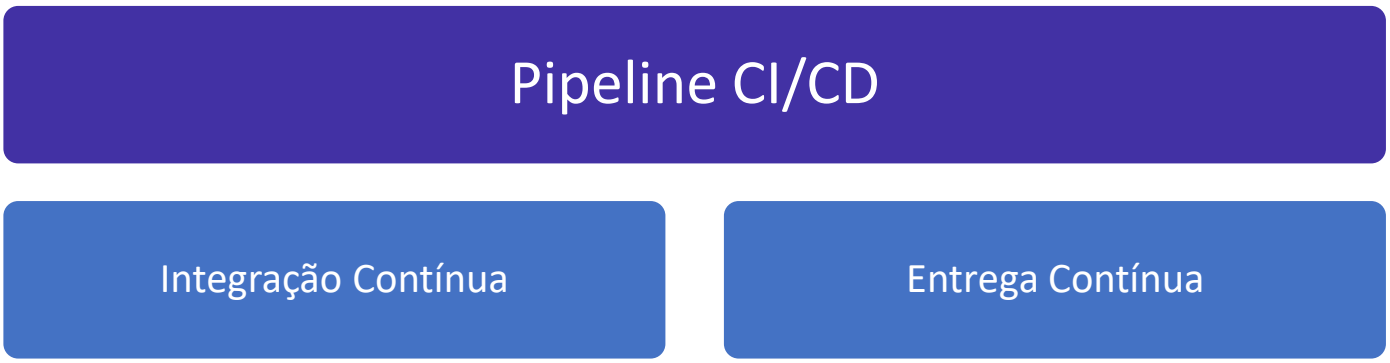
É muito importante entender essa diferença: no modelo tradicional, o QA é uma fase do desenvolvimento; no DevOps, o QA é uma prática contínua que permeia todo o ciclo de vida do software. O profissional de QA deixa de ser apenas um "testador" e passa a ser um engenheiro de qualidade que atua em todas as etapas.

Pipeline CI/CD

O DevOps, acompanhando o desenvolvimento e funcionamento das aplicações, navega através de 8 diferente etapas, conforme cobrado pela banca Cebraspe na prova discursiva para o cargo de Analista do Banco Central, em 2024. Veja só:

- **Planejamento (Plan):** Definição de requisitos, metas e estratégias para o projeto.
- **Código (Code):** Desenvolvimento e versionamento do código-fonte.
- **Build:** Compilação do código em artefatos executáveis.
- **Teste (Test):** Validação automatizada da qualidade e funcionalidade do software.
- **Liberação (Release):** Empacotamento e disponibilização do software para implantação.
- **Implantação (Deploy):** Publicação do software em ambientes de produção ou staging.
- **Operação (Operate):** Gerenciamento e suporte do software em produção.
- **Monitoramento (Monitor):** Observação contínua do desempenho, logs e disponibilidade do sistema.

A **Pipeline CI/CD é um fluxo automatizado de etapas** (ou estágios) para percorrer essas etapas. Ela é divididade em dois “macro grupos”: a **Continuous Integration (CI)**, ou **Integração Contínua**, e a **Continuous Delivery**, ou **Entrega Contínua**. Essas etapas permitem o fluxo contínuo de desenvolvimento de software desde o código-fonte até a entrega final em produção.



O objetivo do pipeline é garantir que cada alteração no código seja validada, testada e implantada de maneira consistente, eficiente e segura, reduzindo a intervenção manual e minimizando erros. Isso é feito por meio de **ferramentas de automação** que gerenciam processos como compilação, testes, validações de qualidade, criação de artefatos e implantação.

Entenda que essa estrutura não é apenas um processo técnico: ele reflete os princípios centrais do DevOps, como **colaboração**, **integração contínua** e **entrega rápida de valor ao cliente**. Ele conecta o trabalho das equipes de desenvolvimento e operações, proporcionando um fluxo contínuo que permite lançar novas versões de software com maior frequência e confiabilidade.

Vamos explorar as duas macro etapas – e uma terceira, chamada de **Implantação Contínua (Continuous Deployment)**, que pode aparecer na sua prova.

Integração Contínua (CI)

A etapa da **Continuous Integration**, ou **Integração Contínua**, carinhosamente chamada de **CI**, se concentra em **integrar frequentemente o código** de diferentes desenvolvedores **em um repositório central**, onde cada alteração é automaticamente verificada por um sistema de automação. O objetivo principal dessa etapa é detectar e corrigir problemas o mais cedo possível, garantindo que o software esteja constantemente em um estado funcional e pronto para avançar para as próximas fases do pipeline CI/CD.

A prática de CI é baseada em processos automatizados que são disparados sempre que uma alteração no código é enviada ao repositório (geralmente por meio de um commit ou pull request). Esses processos incluem:

- **Compilação Automática:** O sistema verifica se o código-fonte pode ser transformado em um artefato funcional, como um binário ou pacote, garantindo que não haja erros básicos de compilação.
- **Testes Automatizados:** Um conjunto de testes (principalmente unitários) é executado para verificar se as alterações introduzidas não quebram funcionalidades existentes.
- **Verificação de Qualidade do Código:** Ferramentas de análise estática, como SonarQube ou ESLint, são usadas para garantir que o código atenda aos padrões de qualidade estabelecidos pela equipe.
- **Relatório de Feedback:** Após a execução dos processos, os resultados (sucesso ou falha) são rapidamente retornados aos desenvolvedores, permitindo uma correção imediata em caso de erro.

A chave do processo de integração é a **automação**. Para isso, uma série de ferramentas são empregadas: Jenkins, Ansible, Puppet, entre outros. Lembre que essa automação percorre todas as etapas – com exceção, talvez, do desenvolvimento do próprio código (o que entra em cheque, considerando a quantidade de ferramentas de IA sendo desenvolvidas para substituir os desenvolvedores, como o Devin).

(CEBRASPE/STJ/2018) Acerca de TomCat, desenvolvimento seguro de software, refactoring e integração contínua, julgue o item.

Em um serviço de integração contínua, testes de unidade são executados automaticamente com a finalidade de detectar erros funcionais.

Comentários:

Isso está correto! Queremos um processo automatizado, o que inclui os testes unitários, aplicados nas menores partes do código, permitindo que sejam detectados erros antes de alguma integração ao repositório central. *(Gabarito: Certo)*

Entrega Contínua (CD)

A **Entrega Contínua**, ou **Continuous Delivery (CD)**, expande os princípios da Integração Contínua (CI), **focando em garantir que o software esteja sempre em um estado pronto para ser implantado** em produção. Ela **se concentra na automação e na padronização do processo de liberação**, eliminando a necessidade de atividades manuais repetitivas e aumentando a confiança na entrega de novas versões do software.

Enquanto a Integração Contínua verifica e valida continuamente o código por meio de compilação e testes, a Entrega Contínua adiciona mais camadas de automação. Isso inclui empacotamento, **testes de integração** e validações de qualidade em um ambiente que simula a produção. A entrega contínua garante que cada alteração aprovada no código esteja pronta para ser liberada, embora a decisão final de implantar em produção (em geral) permaneça como uma ação manual.

A entrega contínua é implementada por meio de pipelines automatizados que executam as seguintes etapas após a integração contínua:

- **Empacotamento:** O software é transformado em artefatos prontos para implantação, como imagens Docker, pacotes .jar/.war ou binários.
- **Testes Funcionais e de Integração:** Além dos testes unitários executados na CI, aqui são realizados testes mais amplos para garantir que os diferentes componentes do sistema funcionam corretamente juntos.
- **Testes de Aceitação do Usuário:** Simulações baseadas em cenários reais para validar se o software atende aos requisitos do cliente.
- **Implantação em Staging:** O software é implantado em um ambiente de staging (ou homologação), que replica a produção, para mais validações.
- **Preparação para Produção:** Após passar por todas as etapas de validação, o software é marcado como pronto para ser implantado em produção.

Implantação Contínua (CD)

A **Implantação Contínua (Continuous Deployment - CD)** é uma prática avançada dentro do ciclo de vida DevOps que **automatiza a entrega de software em produção após passar por todas as etapas de validação**. Diferente da Entrega Contínua (Continuous Delivery), na qual a implantação em produção requer uma aprovação manual, na **Implantação Contínua o processo é totalmente automatizado**. Isso significa que, sempre que uma alteração no código é integrada e validada, ela é automaticamente implantada no ambiente de produção, tornando a nova versão imediatamente disponível para os usuários finais.

A Implantação Contínua exige uma base sólida de automação, incluindo pipelines confiáveis e um conjunto abrangente de testes automatizados, para garantir que o software seja implantado em produção sem riscos significativos. Essa prática é amplamente adotada em organizações que precisam liberar novas funcionalidades, correções e melhorias de maneira frequente e ágil, como serviços SaaS (Software as a Service) – e a escolha entre Entrega Contínua ou Implantação contínua depende de fatores culturais, maturidade do time e necessidades de desenvolvimento.

ENTREGA CONTÍNUA ⟶ INTEGRAÇÃO MANUAL À PRODUÇÃO
IMPLANTAÇÃO CONTÍNUA ⟶ INTEGRAÇÃO AUTOMATIZADA À PRODUÇÃO

(FGV/TJ RJ/2024) O Desenvolvimento de Software tem sido impactado pela condução de projetos empregando métodos ágeis. São alguns dos principais métodos: Scrum; Kanban; Lean; Feature Driven Development (FDD); eXtreme Programming (XP); e, Microsoft Solutions Framework (MSF).

Entre as técnicas aplicadas, destaca-se a prática que torna a integração de código mais eficiente por meio de builds e testes automatizados denominada

a) Iterative Development / Continuous Delivery (ID/CD)
b) Continuous Delivery / Continuous Feedback (CD/CF)
c) Continuous Integration / Continuous Delivery (CI/CD)
d) Flow Production / Continuous Integration (FP/CI)
e) Continuous Integration / Continuous Feedback (CI/CF)

Comentários:

A prática de integração e testes automatizados de código é denominada *Continuous Integration / Continuous Delivery* (CI/CD). (Gabarito: Letra C)

(CEBRASPE/APEX/2024) Em DevOps, a compilação de todos os arquivos de um sistema para a geração de uma versão executável sem nenhum passo manual é denominada

a) servidores de integração contínua.
b) deploy contínuo.
c) build automatizado.
d) testes automatizados.

Comentários:

Vamos analisar cada alternativa.

a) Errado. Os servidores de integração contínua São ferramentas que gerenciam pipelines, mas não descrevem o processo específico de compilação.
b) Errado. O deploy contínuo refere-se à implantação automatizada de alterações no ambiente de produção, o que ocorre após o build e validação.
c) Certo. O build automatizado é o processo em que todos os arquivos de código-fonte de um sistema são compilados automaticamente para gerar uma versão executável (artefato), sem intervenção manual.
d) Errado. Os testes são etapas de validação, não o processo de compilação ou geração de artefatos.

Portanto, correta a letra C. *(Gabarito: Letra C)*

CALMS DevOps

O **CALMS DevOps** é um modelo utilizado para avaliar e guiar a adoção da cultura e das práticas do **DevOps** em uma organização. Ele se baseia em cinco pilares fundamentais: **Cultura (Culture)**, **Automação (Automation)**, **Lean**, **Medição (Measurement)** e **Compartilhamento (Sharing)**. Esse modelo não é um framework técnico, mas sim uma abordagem para **alinhar equipes e processos com os princípios do DevOps**, ajudando na transformação organizacional e no sucesso das iniciativas.



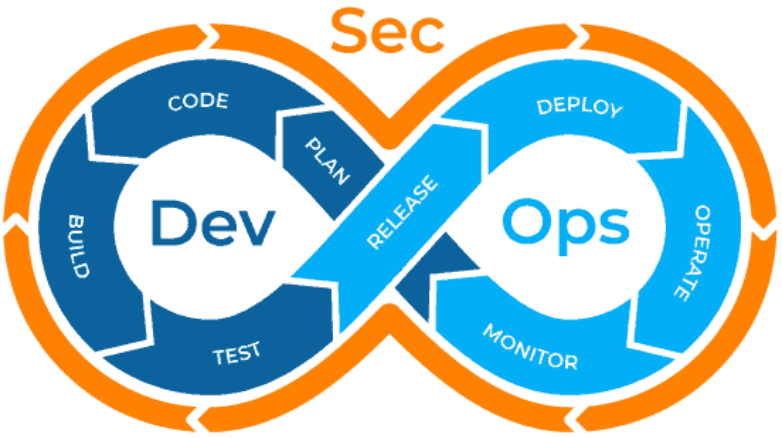
CALMS	Descrição
Culture	Promove um ambiente colaborativo, transparente e de confiança entre equipes, reduzindo silos e priorizando a entrega de valor ao cliente em vez de responsabilidades isoladas.
Automation	Automatiza tarefas manuais e repetitivas, como integração, entrega, provisionamento de infraestrutura e testes, aumentando a consistência, eficiência e velocidade no ciclo de entrega de software.
Lean	Baseado nos princípios do Lean Manufacturing, foca na eliminação de desperdícios, otimização de processos, ciclos curtos de entrega e foco no que realmente agrega valor ao cliente.
Measurement	Coleta e analisa métricas para monitorar desempenho, identificar problemas e tomar decisões orientadas por dados, incluindo métricas de equipe, técnicas e de negócio.
Sharing	Incentiva a troca de informações e aprendizado entre equipes, promovendo transparência, reuniões pós-incidente, documentação aberta e uma cultura de aprendizado coletivo.

O CALMS serve como um framework conceitual para organizações avaliarem sua maturidade em DevOps. Ele não é uma receita rígida, mas uma forma de identificar áreas de melhoria e alinhar práticas ao objetivo maior de entrega de valor contínuo.

- Cultura: Está promovendo colaboração entre equipes?
- Automação: Seus processos estão suficientemente automatizados para evitar retrabalho manual?
- Lean: Os processos são eficientes ou há desperdícios?
- Medição: Você está usando dados para orientar decisões?
- Compartilhamento: Existe troca de informações e aprendizado coletivo?

DevSecOps

DevSecOps, ou **Devlopment Security Operations** (Operações de Segurança no Desenvolvimento) envolve a prática de **integrar testes de segurança** a **TODAS** as etapas do processo de desenvolvimento de software, incluindo ferramentas e processos que incentivem a **colaboração entre membros da equipe**.



Aqui ocorre uma sobreposição tripla da equipe, temos os desenvolvedores, os especialistas em infraestrutura e os especialistas em segurança. Então, tudo que vimos sobre DevOps soma-se ao que vimos em aulas sobre segurança. A grande chave do DevSecOps é um assunto que já vimos lá atrás é o emprego de testes para a garantia de segurança em **todas as etapas do ciclo de vida** – uma abordagem baseada no Ciclo de Desenvolvimento Seguro:



Então, é importante que você saiba diferenciar o DevOps do DevSecOps – e saber que ambos trabalham em conjunto. Para auxiliar, vou trazer um texto na íntegra da Amazon Web Services (AWS):



O DevOps se concentra em colocar uma aplicação no mercado o mais rápido possível. No DevOps, o teste de segurança é um processo separado que ocorre no final do desenvolvimento da aplicação, pouco antes de ser implantada. Normalmente, uma equipe separada testa e aplica a segurança ao software. Por exemplo, as equipes de segurança configuram um firewall para testar a invasão na aplicação depois que ela foi criada.

***No DevSecOps**, por sua vez, os **testes de segurança passam a fazer parte do próprio processo de desenvolvimento** de aplicações. As equipes de segurança e desenvolvedores trabalham juntos para proteger os usuários contra vulnerabilidades de software. Por exemplo, as equipes de segurança configuram firewalls, os programadores projetam o código para evitar vulnerabilidades, e os testadores testam todas as alterações para impedir acesso não autorizado de terceiros.*

(CESPE/TRT 8/2022) A respeito de testes automatizados, no contexto de DevOps e DevSecOps, assinale a opção correta.

a) Em um teste unitário, os métodos da classe sendo testada e suas dependências podem ter relação com recursos externos.
b) Os bugs são detectados no final do ciclo de desenvolvimento, o que pode aumentar o tempo na criação de novos produtos.
c) Os testes unitários são testes de caixa preta com cada função que compõe o software.
d) O TDD (*Test Driven Development*) eleva o nível dos testes unitários e tem como característica criar a classe de testes antes da classe de produção, de forma que os testes guiem o código a ser implementado.
e) Os testes de integração são caracterizados pela verificação de partes internas do sistema, que se inter-relacionam entre si, conforme definido pelos clientes.

Comentários:

Como vamos ver nessa questão, as questões que abordam DevSecOps nada mais são que questões de segurança da informação e engenharia de software aplicadas. Vamos analisar cada afirmativa.

a) Errado. Galera, testes unitários testam partes autônomas do programa e exaurem os testes em si – ou seja, não temos relações com recursos externos.
b) Errado. No DevSecOps temos testes constantes, permitindo a detecção de *bugs* em diversas fases do desenvolvimento.
c) Errado. Testes unitários são testes de caixa branca, já que estamos testando a funcionalidade e funcionamento do código em si.
d) Certo. No TDD temos um aumento na quantidade de testes, já que desenvolvemos de acordo com os testes, que foram planejados em etapa anterior ao desenvolvimento.
e) Errado. A integração é caracterizada por verificar o funcionamento em conjunto de componentes, se relacionando.

Portanto, correta a letra D. *(Gabarito: Letra D)*

(CEBRASPE/BACEN/2024) Acerca dos princípios e das práticas de DevOps e DevSecOps, integração e entrega contínuas, julgue os seguinte o item.

02648216103 - HENRIQUE AMORIM BARRETO

Integração, entrega e implantação contínuas e com segurança em todo o ciclo de vida do desenvolvimento de *software* são os princípios básicos do DevOps.

Comentários:

A questão parece "certinha" – mas quem foca em ter *"segurança em todo o ciclo de vida do desenvolvimento"* é o DevSecOps. Embora o DevOps implemente testes em todo o ciclo de desenvolvimento, eles são voltados para a integridade das implementações e na continuidade de funcionamento da aplicação. *(Gabarito: Errado)*

QUESTÕES COMENTADAS

01. (FGV/TJ RJ/2024) O Desenvolvimento de Software tem sido impactado pela condução de projetos empregando métodos ágeis. São alguns dos principais métodos: Scrum; Kanban; Lean; *Feature Driven Development* (FDD); *eXtreme Programming* (XP); e, *Microsoft Solutions Framework* (MSF).

Entre as técnicas aplicadas, destaca-se a prática que torna a integração de código mais eficiente por meio de *builds* e testes automatizados denominada

a) *Iterative Development* / *Continuous Delivery* (ID/CD)
b) *Continuous Delivery* / *Continuous Feedback* (CD/CF)
c) *Continuous Integration* / *Continuous Delivery* (CI/CD)
d) *Flow Production* / *Continuous Integration* (FP/CI)
e) *Continuous Integration* / *Continuous Feedback* (CI/CF)

Comentários:

A estrutura que irá aplicar a integração automatizada é a *CI/CD – Continuous Integration & Continuous Delivery*.

Gabarito: Letra C

---

02. (CEBRASPE/CNPq/2024) Julgue o item subsequente, com relação a ferramentas de DevOps, gestão de ativos e autenticação única (*single sing-on*).

Em DevOps, a análise estática de código e geração de documentação fazem parte do processo de *build* de uma aplicação.

Comentários:

O item está correto. O processo de build de uma aplicação pode incluir etapas além da simples compilação do código, como a análise estática e a geração de documentação. A análise estática de código verifica o código-fonte em busca de erros, vulnerabilidades ou violações de padrões de qualidade, antes mesmo de sua execução. Isso é essencial para garantir a qualidade e segurança do software.

Já a geração de documentação automática durante o build é uma prática comum em pipelines de Integração Contínua (CI), onde ferramentas como Javadoc, Sphinx ou Doxygen geram documentação técnica baseada em anotações ou comentários no código. Essas práticas ajudam a

padronizar e melhorar o ciclo de desenvolvimento, alinhando-se aos princípios do DevOps de automação e qualidade contínuas.

Gabarito: Certo

---

03. (CEBRASPE/CNPq/2024) Julgue o item subsecutivo, relacionado a DevSecOps.

No contexto das equipes de desenvolvimento de software, o DevSecOps representa uma evolução pelo fato de integrar ao processo uma equipe específica que trata exclusivamente da segurança de software, liberando as equipes de desenvolvimento e operacionais dessa tarefa, de forma a agilizar significativamente a entrega do software.

Comentários:

O DevSecOps não se baseia em delegar exclusivamente a segurança para uma equipe separada, mas sim em integrar a segurança ao longo de todo o ciclo de vida do desenvolvimento de software. O objetivo do DevSecOps é tornar a segurança uma responsabilidade compartilhada entre todas as equipes — desenvolvimento, operações e segurança — desde o início do processo. Ao invés de isolar a segurança como responsabilidade de uma equipe específica, o DevSecOps incorpora práticas como análises de vulnerabilidades automatizadas, testes de segurança contínuos e treinamento para desenvolvedores.

Gabarito: Errado

---

04. (CEBRASPE/CNPq/2024) Acerca de conteinerização e de DevOps, julgue o item subsequente.

O conceito de DevOps envolve a automatização de processos e colaboração entre equipes.

Comentários:

Questão tranquilinha – de fato, o DevOps envolve esses fatores, buscando um processo integrado e automatizado de desenvolvimento e operação.

Gabarito: Certo

---

05. (FGV/DNIT/2024) DevOps é a fusão das primeiras letras de duas funções distintas dentro da TI: desenvolvedores (responsáveis por escrever software) e a equipe de operações (responsável por manter a infraestrutura na qual o software é desenvolvido e implantado). Já o DevSecOps, é uma combinação das primeiras letras de “desenvolvimento”, “segurança” e “operações”.

A esse respeito, analise os itens a seguir.

> I. DevSecOps é uma extensão da mudança cultural em direção ao DevOps, incorporando todos dentro da organização, incluindo partes interessadas nos negócios, segurança, arquitetos, proprietários de produtos e engenheiros de desenvolvimento e de operações.
> II. DevOps é a colaboração de pessoal e processos de desenvolvimento e operações para formar uma única equipe de entrega ágil composta por partes interessadas, clientes, engenheiros e testadores. Essa equipe trabalha em conjunto para fornecer software e infraestrutura em ritmo acelerado por meio de um pipeline automatizado de CI/CD.
> III. É natural supor que o DevSecOps una engenheiros de requisitos, de dados e de redes com DevOps em uma única equipe. No entanto, a segurança é criada por um grupo multidisciplinar de indivíduos externos à organização, cada um com uma função específica, testar falhas e explorar vulnerabilidades em sistemas e bancos de dados.

Está correto o que se afirma em

a) I, apenas.
b) III, apenas.
c) I, II e III.
d) I e II, apenas.
e) II e III, apenas.

Comentários:

Vamos analisar os itens.

I. Certo. O DevSecOps é, de fato, uma extensão do DevOps que adiciona a preocupação com a segurança em todas as etapas do ciclo de vida do desenvolvimento. Ele não se limita apenas às equipes técnicas (desenvolvedores e operações), mas também envolve outras partes interessadas, como segurança, negócios, arquitetos e proprietários de produtos, promovendo uma abordagem colaborativa e cultural.
II. Certo. O DevOps enfatiza a colaboração entre equipes de desenvolvimento e operações, formando uma equipe única que trabalha em conjunto para entregar software e infraestrutura de forma ágil e contínua. A adoção de práticas como pipelines automatizados de CI/CD reflete essa filosofia, permitindo entregas rápidas e confiáveis.
III. Errado. Embora o DevSecOps una diferentes áreas e promova a colaboração entre engenheiros de requisitos, dados, redes e segurança, a segurança não é responsabilidade exclusiva de indivíduos externos à organização. A ideia do DevSecOps é justamente integrar a segurança diretamente no processo interno de desenvolvimento e operações, tornando-a uma responsabilidade compartilhada entre todas as equipes. Grupos externos podem realizar

auditorias ou testes específicos (como pen tests), mas a segurança não é exclusivamente delegada a eles.

Portanto, correta a letra D.

Gabarito: Letra D

---

06. (FGV/DNIT/2024) Infraestrutura como código (IaC) é uma forma de gerenciamento e provisionamento de infraestrutura por meio de código em vez de processos manuais.

Com relação às três práticas para implementar IaC, analise as afirmativas a seguir.

I. Definir todas as suas coisas como código é uma prática básica para fazer alterações rápidas e de modo confiável. Os principais motivos para a adoção dessa prática são a interoperabilidade, a consistência e a acessibilidade.
II. Testar e entregar continuamente todo o trabalho em andamento é uma prática onde as equipes de infraestrutura são rigorosas nos testes. Eles usam a automação para implantar e testar o sistema como um todo antes de testar e integrar todos os módulos desenvolvidos por todos os envolvidos.
III. Construir peças pequenas e simples, que os membros da equipe podem alterar independentemente é uma prática essencial, pois muitas equipes enfrentam dificuldades quando seus sistemas são grandes e fortemente acoplados. Quanto maior um sistema, mais difícil mudá-lo e mais fácil quebrar.

Está correto o que se afirma em

a) I e II, apenas.
b) I, apenas.
c) II, apenas.
d) III, apenas.
e) II e III, apenas.

Comentários:

Vamos aos itens.

I. Errado. Embora definir toda a infraestrutura como código seja uma prática essencial no IaC, os benefícios apontados estão incorretos. Os principais motivos para adotar essa prática são reprodutibilidade, rastreabilidade e facilidade de automação, enquanto "interoperabilidade, consistência e acessibilidade" não são os principais objetivos do IaC.
II. Errado. No contexto do IaC, a entrega e os testes contínuos normalmente seguem uma abordagem que integra e testa cada módulo individualmente antes de validar o sistema como um

todo. Além disso, a integração e os testes são feitos em paralelo, e não sequencialmente como descrito.
III. Certo. Construir sistemas modulares, pequenos e independentes é uma prática essencial no IaC. Isso permite que mudanças sejam realizadas de forma mais simples e com menor risco, reduzindo a complexidade do sistema como um todo. Sistemas grandes e acoplados dificultam a manutenção e aumentam a probabilidade de falhas.

Sendo assim, correta a letra D.

Gabarito: Letra D

---

07. (CEBRASPE/INPI/2024) A respeito de metodologias e técnicas prescritas pela engenharia de *software* para o desenvolvimento e para a gestão de produtos, julgue o item que se segue.

Na entrega de *software* funcionando para a produção de forma ágil, o DevOps prestigia a entrega contínua; nesse sentido, um princípio recomendado é que as atualizações de arquivo do servidor, por meio de uma operação de atualização de arquivos (commit) realizada pelos desenvolvedores, sejam centralizadas em uma máquina de integração.

Comentários:

É uma prática comum que as atualizações realizadas pelos desenvolvedores (commits) sejam centralizadas em uma máquina de integração ou um servidor de integração contínua. Esse servidor automatiza processos como compilação, execução de testes e validação do código, garantindo que o sistema esteja em um estado funcional após cada integração.

Essa prática promove:

- Centralização do controle: Todas as alterações são reunidas em um repositório central, onde o pipeline CI/CD pode verificar sua consistência.
- Automação: Processos manuais, como compilação e testes, são substituídos por automação.
- Detecção precoce de erros: Problemas no código são identificados rapidamente, antes de serem propagados para outros desenvolvedores ou etapas do pipeline.

Portanto, o item está correto.

Gabarito: Certo

---

08. (CEBRASPE/APEX/2024) Em uma arquitetura DevOps, existem diferenças entre a integração contínua e o *deployment* contínuo. Assinale a opção em que é apresentada característica aplicada ao *deployment* contínuo.

a) Torna novas entregas um não evento (não existindo mais um “dia D”), evitando, assim, que a perda de um *deadline* atrase a entrega de uma funcionalidade por meses.
b) *Builds* e testes devem ser executados com frequência, se possível após cada *commit* realizado no *master*.
c) É recomendável para sistemas *desktop*, *app* móvel ou embarcado que precisem de atualização.
d) Aumenta o tempo de entrega de novas funcionalidades, que são liberadas assim que ficam prontas, diminuindo o intervalo entre *releases* e o número de *releases*.

Comentários:

Vamos analisar as alternativas.

a) Certo. O Deployment Contínuo (Continuous Deployment) automatiza todo o pipeline de entrega, incluindo a implantação de alterações diretamente no ambiente de produção sem intervenção manual. Essa prática elimina o conceito de "dia D" (grandes eventos de liberação), tornando as entregas frequentes, regulares e previsíveis. Assim, cada alteração aprovada no pipeline é liberada automaticamente, sem a necessidade de agendamento ou aprovação final.
b) Errado. Essa descrição aplica-se à Integração Contínua (Continuous Integration), onde o foco está em verificar frequentemente as alterações do código.
c) Errado. Embora o deployment contínuo possa ser aplicado nesses sistemas, ele não é uma característica exclusiva ou recomendável apenas para eles.
d) Errado. O deployment contínuo reduz o tempo de entrega, permitindo que as funcionalidades sejam liberadas com frequência, aumentando o número de releases.

Portanto, correta a letra A.

Gabarito: Letra A

---

09. (CEBRASPE/MPO/2024) Acerca do DevOps, julgue o item seguinte.

Os recursos de monitoramento são de responsabilidade privativa, nessa abordagem, da equipe de operações no processo de CI/CD (continuous integration / continuous delivery).

Comentários:

Um dos pilares do DevOps é justamente a integração, visando acabar com os silos organizacionais – ou seja, não teremos um especialista isolado realizando cada etapa, e sim equipes multidisciplinares. Então não faz sentido o monitoramento ter uma responsabilidade privativa.

Gabarito: Errado

---

10. (CEBRASPE/BACEN/2024) A respeito de infraestrutura como código, Docker e Kubernetes, julgue o item a seguir.

No modelo de infraestrutura como código, é possível a implantação de uma infraestrutura com definição de sub-redes, balanceadores de carga e máquinas virtuais com a utilização de código, sem a necessidade de configurações manuais.

Comentários:

O item está correto. O modelo de Infraestrutura como Código (IaC) permite a definição, configuração e implantação de recursos de infraestrutura utilizando código declarativo ou scripts automatizados. Isso inclui a criação de sub-redes, balanceadores de carga e máquinas virtuais, entre outros elementos, sem a necessidade de configurações manuais diretas na interface gráfica.

Gabarito: Certo

---

11. (FGV/PGM NITERÓI/2023) Um Time de Desenvolvimento de Software (TDS) segue um protocolo automatizado para gerar, testar e combinar pacotes de software gerados separadamente. Todo software combinado precisa passar por um processo que inclui uma requisição formal ao Time de Operações (TO) de um Centro de Dados para executar um conjunto de testes, com o intuito de verificar vulnerabilidades no software antes de entrar em produção.

Considerando os conceitos de DevOps e DevSecOps, o TDS e o TO estão falhando no princípio:

a) automação;
b) colaboração ágil;
c) produção contínua;
d) integração contínua;
e) ação centrada no cliente.

Comentários:

O defeito na pipeline de desenvolvimento do software é esse aqui: *"precisa passar por um processo que inclui uma requisição formal ao Time de Operações (TO) de um Centro de Dados para executar um conjunto de testes"*

Veja que os testes não estão sendo aplicados de forma automática. Então, a falha é no princípio da automação.

Gabarito: Letra A

02648216103 - HENRIQUE AMORIM BARRETO

12. (CEBRASPE/SERPRO/2023) Acerca da gestão de configuração, julgue o item a seguir.

A prática de database migration é fundamental em DevOps para garantir a consistência dos dados bem como sua disponibilidade e segurança.

Comentários:

Database Migration, ou migração de banco de dados, segundo o Google Cloud, é:

"Migração de banco de dados é o uso de um serviço de migração de banco de dados para migrar dados de um banco de origem para um ou mais bancos de destino. Quando uma migração é concluída, o conjunto de dados nos bancos de dados de origem passa a residir totalmente no banco de destino, embora possivelmente de maneira reestruturada. Os clientes que acessaram os bancos de dados de origem são transferidos para os bancos de destino e os bancos de origem são desativados".

Essa é uma prática recorrente em aplicações modernas e merece bastante atenção, já que ela permite oferecer dados com consistência e alta disponibilidade.

Gabarito: Certo

---

13. (CEBRASPE/EMPREL/2023) No DevOps, existe uma prática em que novas alterações de código de uma solução de software são criadas, testadas e mescladas regularmente em um repositório central, com objetivo de encontrar e investigar bugs mais rapidamente. Trata-se da

a) integração contínua.
b) entrega contínua.
c) implantação em contêineres.
d) disponibilização em microsserviços.
e) pipeline.

Comentários:

Quando estamos "mesclando alterações de código em um repositório central", essa prática refere-se à integração contínua.

Gabarito: Letra A

---

14. (CEBRASPE/PREF. FORTALEZA/2023) Tendo em vista que, no atual cenário de desenvolvimento de aplicações web, é essencial considerar princípios, como consistência e

escalabilidade, e práticas, como automação do processo de implantação e integração do código-fonte, julgue o item subsequente.

As ferramentas DevOps incluem o Kubernetes, uma plataforma de orquestração de contêineres que permite gerenciar e escalonar aplicativos em contêineres em diferentes ambientes e pode ser integrada a outras ferramentas DevOps, como Docker e Jenkins.

Comentários:

Com os conceitos introdutórios dos programas, você já consegue responder a essa questão. Kubernetes é uma das principais ferramentas no DevOps, responsável por fazer orquestração de contêiners. Esses contêiners podem ser criados pelo Docker, e podem ter tarefas automatizadas com o Jenkins. Então, correta a afirmativa.

Gabarito: Certo

---

15. (CEBRASPE/PREF. FORTALEZA/2023) Julgue o item que se segue, com relação a técnicas de integração e implantação contínua de código.

Embora as técnicas de integração contínua sejam essenciais para garantir a qualidade do software e a eficiência do processo de desenvolvimento, elas não possibilitam a realização de testes automatizados de aceitação.

Comentários:

Errado! A integração contínua procura justamente a automatização – o que se aplica também aos testes. Então, é errado afirmar que não possibilita a realização de testes automatizados.

Gabarito: Errado

---

16. (CEBRASPE/MPE RO/2023) Determinada empresa adota a cultura DevOps, com os objetivos de colaboração e integração do desenvolvimento de software e das operações. A mesma empresa também adota práticas de automação em que alterações de código são frequentes e adicionadas a um repositório central de versionamento, para que, em seguida, ocorram as etapas de build e teste para melhorar a qualidade.

Nessa situação hipotética, a empresa aplica o conceito de

a) agile scrum.
b) continuous integration.
c) machine learning.

d) blockchain.
e) continuous delivery.

Comentários:

Novamente, acredito que você já deva ter encontrado o padrão nas questões. Quando a banca afirma: *"em que alterações de código são frequentes e adicionadas a um repositório central de versionamento"*. Sempre que tivermos algo relacionado a implementação no repositório central, temos a integração contínua.

Gabarito: Letra B

---

17. (CEBRASPE/DATAPREV/2023) A respeito de DevSecOps e de técnicas de análise de desempenho, julgue o item a seguir.

O DevSecOps deve evitar que o ciclo de trabalho do DevOps fique muito lento, observando sempre a segurança da aplicação e a infraestrutura, e, para isso, um dos cuidados que se deve ter é evitar automatizar barreiras de segurança, o que pode inserir pontos de fragilidade no software, comprometendo a segurança e o tempo de desenvolvimento.

Comentários:

O DevSecOps deve, de fato, evitar que sejam implementadas barreiras demasiadas, de forma a tornar lento demais o ciclo de desenvolvimento – o ciclo irá desacelerar, obviamente, mas nada em ponto crítico. Mas a afirmativa erra ao dizer que temos que evitar automatizar, sendo que devemos justamente procurar a automatização, que dá mais confiança e celeridade ao processo.

Gabarito: Errado

---

18. (CEBRASPE/DATAPREV/2023) A respeito de DevSecOps e de técnicas de análise de desempenho, julgue o item a seguir.

O DevSecOps trata de segurança integrada, e não da segurança que funciona no perímetro em torno de aplicativos e dados, a qual também inclui a identificação dos riscos que estão envolvidos na cadeia de fornecimento do software.

Comentários:

Perfeito! A DevSecOps é uma prática integrada, aplicando os preceitos da segurança da informação ao desenvolvimento, e não encarando ferramentas de segurança pontuais, que ficam a encargo de setores específicos.

Gabarito: Certo

---

19. (CEBRASPE/DATAPREV/2023) Tendo como referência conceitos e princípios de DevOps e DevSecOps, julgue o item a seguir, relativo aos 6 Cs.

O processo de entrega contínua envolve a automação de processos de desenvolvimento, testes e integração dos códigos de forma frequente e em um repositório compartilhado.

Comentários:

Galera, quem faz a automação dos processos e a integração de código ao repositório é a integração contínua, não a entrega contínua.

Gabarito: Errado

---

20. (CEBRASPE/PETROBRAS/2022) Julgue o item subsecutivo, relativo a DevOps e notação BPMN.

No DevOps, a integração contínua possui como uma de suas atividades a realização de testes; a fim de se obter os benefícios esperados convém automatizar os testes para poder executá-los para cada alteração feita no repositório principal.

Comentários:

Perfeito! Venho ressaltando isso para vocês a aula toda: a chave do DevOps é a automação. E é a automação de TUDO, principalmente dos testes.

Gabarito: Certo

---

21. (FCC/PGE AM/2022) A transição de DevOps para DevSecOps requer a compreensão e utilização de técnicas e práticas específicas que podem garantir a segurança do software. Uma especialista em Engenharia de Software recomendou, dentre outras, as seguintes ferramentas e tecnologias para essa transição em uma empresa:

> I. É usado para verificar o código sem realmente executá-lo. Este tipo de ferramenta ajuda a encontrar vulnerabilidades em potencial no código-fonte, evitando que ocorram várias vulnerabilidades do tipo zero-day. Common Weakness Enumeration (CWE) é uma das classificações de avisos mais comuns produzidos por estas ferramentas. CWE é uma lista

oficial ou dicionário de pontos fracos de segurança comuns exploráveis por invasores para obter acesso não autorizado ao sistema.
II. Da mesma forma que as ferramentas que executam testes de caixa preta, estes analisadores dinâmicos podem identificar vulnerabilidades do programa, como injeções de SQL, estouros de buffer e similares.
III. Este tipo de ferramenta analisa o comportamento do aplicativo, implementando uma análise de segurança contínua, sendo uma das tecnologias de segurança usadas em tempo de execução.

Os itens I, II e III correspondem, correta e respectivamente, a

a) RASP (Runtime Application Self-Protection) – SAST (Static Application Security Testing) – IAST (Interactive Application Security Testing).
b) IAST (Interactive Application Security Testing) – SAST (Static Application Security Testing) – DAST (Dynamic Application Security Testing).
c) SAST (Static Application Security Testing) – IAST (Interactive Application Security Testing) – DAST (Dynamic Application Security Testing).
d) IAST (Interactive Application Security Testing) – SAST (Static Application Security Testing) – RASP (Runtime Application Self-Protection).
e) SAST (Static Application Security Testing) – DAST (Dynamic Application Security Testing) – RASP (Runtime Application Self-Protection).

Comentários:

• IAST – Interactive Application Security Testing: uma técnica de testagem de segurança que analisa aplicações em tempo real, identificando e reportando vulnerabilidades enquanto a aplicação está sendo executada.
• DAST – Dynamica Application Security Testing: uma abordagem que avalia a segurança de uma aplicação em execução, simulando ataques e identificando vulnerabilidades com base na interação dinâmica com a aplicação.
• SAST – Static Application Security Testing: uma técnica de testagem de segurança que avalia o código-fonte de uma aplicação em um estado estático, sem a necessidade de executar o software.
• RASP – Runtime Application Self-Protection: solução de segurança que é integrada diretamente à aplicação e monitora sua execução em tempo real, identificando e respondendo automaticamente a potenciais ameaças.

Então, vamos analisar cada item.

I. Quando analisamos sem executar o código, estamos fazendo uma análise estática (SAST).
II. As verificações dinâmicas, focadas em detectar bugs, ocorrem com o código em funcionamento – e é encargo dos DAST.

III. Quando focamos na análise de segurança contínua, em tempo real, estamos falando do RASP.

Portanto, correta a letra E.

Gabarito: Letra E

---

22. (FCC/TRT 4/2022) Integração e implantação contínuas (CI/CD) é uma coleção de práticas usadas para integrar e entregar código rapidamente. Usando a prática de

a) integração contínua (CI), em um cronograma pré-acordado, provavelmente no final do ciclo de sprint, o pacote de código é liberado e testado manualmente pelo time de desenvolvimento.
b) implantação contínua (CD), depois que todo o código é validado com sucesso pelo time de desenvolvimento e qualidade, implanta-se o código manualmente no ambiente de teste ou de produção.
c) implantação contínua (CD) em que o grupo controle de qualidade realiza testes manuais para todos os códigos novos ou modificados. Se for bem-sucedido, o código será liberado para implantação; caso contrário, o lançamento é rejeitado e colocado em espera até que os bugs sejam corrigidos.
d) integração contínua (CI), os desenvolvedores são incentivados a fazer pequenos ajustes de código, mesclar seu código na ramificação de código principal e validá-lo usando testes automatizados.
e) integração contínua (CI), para implementar pipelines devem ser utilizadas ferramentas como Zuul e Eureka.

Comentários:

Vamos analisar cada afirmativa.

a) Errado. Galera, falou em CI e teste manual, já está errado.
b) Errado. Novamente, viu manualmente, errado.
c) Errado. Outra vez, testes manuais.
d) Certo. Perfeito, a implementação deve se dar de forma automatizada.
e) Errado. Zuul e Eureka são ferramentas da Netflix, focadas na IoT.

Correta, portanto, a letra D.

Gabarito: Letra D

---

23. (FGV/TRT 16/2022) Considerando o DevOps e suas boas práticas, analise os itens a seguir:

02648216103 - HENRIQUE AMORIM BARRETO

I. Testes integrados são uma parte importante do processo DevOps. Esses testes devem levar em consideração as práticas de Test-Driven Development e Behavior-Driven Development, dessa forma a execução automática desses testes pode ser integrada ao pipeline de CI. No entanto, é importante integrar outros tipos de testes, como testes funcionais ou testes de integração, que permitem que o aplicativo seja testado funcionalmente do início ao fim com os outros componentes do seu ecossistema.
II. Recomenda-se automatizar apenas as tarefas críticas que envolvam poucas atualizações na implementação e nos testes dos aplicativos nas infraestruturas. Essas tarefas devem ser automatizadas em scripts que podem ser facilmente integradas e executadas em pipelines de CI/CD.
III. A construção de pipelines de CI/CD envolvem a escolha de ferramentas de DevOps adequadas pelas equipes considerando a natureza da empresa. É necessário levar em conta aspectos financeiros, avaliar entre ferramentas de código aberto e gratuitas e as proprietárias, que são mais ricas em recursos e suporte, mas exigem um investimento significativo.

Está correto apenas o que se afirma em

a) I e III.
b) I.
c) I e II.
d) II e III.
e) II.

Comentários:

Vamos ver cada afirmativa.

I. Certo. Perfeito, galera. A automatização envolvem todos os tipos de teste – unitários, de integração, funcionais etc.
II. Errado. As ferramentas devem ser propagadas para todo o aplicativo, não só partes críticas.
III. Certo. Galera, existe uma infinidade de programas e ferramentas de DevOps. Cada entidade deve estudar as que são aplicáveis à sua situação, seja por limites financeiros, de estrutura, ou quaisquer outros.

Correta, portanto, a letra A.

Gabarito: Letra A

---

24. (CEBRASPE/ALECE/2021) DevOps, termo criado para descrever um conjunto de práticas para integração entre as equipes de desenvolvimento de softwares, tem como princípio

a) utilizar padrões de projeto para problemas recorrentes e conhecidos no desenvolvimento de software.
b) implantar a solução de software por meio de processos repetíveis e confiáveis.
c) desenvolver o software por meio funcionalidades implementadas e disponibilizadas na forma de serviços.
d) desenvolver soluções em no mínimo três camadas: de interação (usuário), de manipulação dos dados (modelo) e de controle.
e) utilizar serviço de computação baseado na Internet por meio de recursos de TI sob demanda.

Comentários:

Vamos analisar cada uma das afirmativas.

a) Errado. Embora seja uma prática recomendada para o desenvolvimento de softwares, ela não é associada diretamente a DevOps.
b) Certo. Exatamente isso que o DevOps prega – implementar soluções que possam ser repetíveis e forneçam resultados confiáveis.
c) Errado. Isso é arquitetura orientada a serviços (OSS), não DevOps.
d) Errado. Não existe essa delimitação de camadas.
e) Errado. Embora seja recomendado o uso de computação baseada na internet (mais especificamente, na nuvem), isso pode não ser aplicado a todos.

Portanto, correta a letra B.

Gabarito: Letra B

---

25. (FCC/TRF 4/2019) Um Analista de TI, ao pesquisar sobre testes automatizados, encontrou a seguinte informação:

Para escrever os testes de aceitação, podemos usar os critérios de aceitação descritos nas histórias de usuário. Estando estes testes implementados, executando e passando com sucesso, obter-se-á uma boa cobertura e garantia de que pelo menos as funcionalidades mais importantes estão sendo de fato cobertas pelos testes. É recomendável que esses testes sejam executados por um tipo de ferramenta (como: Jenkins, TravisCI, Hudson, CruiseControl etc.), que fará o trabalho de executar os testes automatizados de acordo com alguma estratégia predefinida (como: todo dia, a cada modificação feita no código-fonte etc.).

O Analista conclui que o tipo de ferramenta mencionada no texto é de

a) teste unitário.
b) integração contínua.

c) teste de usabilidade.
d) teste cross-platform.
e) DDT- Database Driven Testing.

Comentários:

A afirmação da questão está envolvendo a automação do processo de desenvolvimento, com destaque para os testes. Isso se destaca pelas ferramentas indicadas, como o Jenkis e TravisCI, que compõem o ecossistema DevOps. Portanto, estamos falando da integração contínua.

Gabarito: Letra B

02648216103 - HENRIQUE AMORIM BARRETO

LISTA DE QUESTÕES

01. (FGV/TJ RJ/2024) O Desenvolvimento de Software tem sido impactado pela condução de projetos empregando métodos ágeis. São alguns dos principais métodos: Scrum; Kanban; Lean; *Feature Driven Development* (FDD); *eXtreme Programming* (XP); e, *Microsoft Solutions Framework* (MSF).

Entre as técnicas aplicadas, destaca-se a prática que torna a integração de código mais eficiente por meio de *builds* e testes automatizados denominada

a) *Iterative Development* / *Continuous Delivery* (ID/CD)
b) *Continuous Delivery* / *Continuous Feedback* (CD/CF)
c) *Continuous Integration* / *Continuous Delivery* (CI/CD)
d) *Flow Production* / *Continuous Integration* (FP/CI)
e) *Continuous Integration* / *Continuous Feedback* (CI/CF)

02. (CEBRASPE/CNPq/2024) Julgue o item subsequente, com relação a ferramentas de DevOps, gestão de ativos e autenticação única (*single sing-on)*.

Em DevOps, a análise estática de código e geração de documentação fazem parte do processo de *build* de uma aplicação.

03. (CEBRASPE/CNPq/2024) Julgue o item subsecutivo, relacionado a DevSecOps.

No contexto das equipes de desenvolvimento de software, o DevSecOps representa uma evolução pelo fato de integrar ao processo uma equipe específica que trata exclusivamente da segurança de software, liberando as equipes de desenvolvimento e operacionais dessa tarefa, de forma a agilizar significativamente a entrega do software.

04. (CEBRASPE/CNPq/2024) Acerca de conteinerização e de DevOps, julgue o item subsequente.

O conceito de DevOps envolve a automatização de processos e colaboração entre equipes.

05. (FGV/DNIT/2024) DevOps é a fusão das primeiras letras de duas funções distintas dentro da TI: desenvolvedores (responsáveis por escrever software) e a equipe de operações (responsável por manter a infraestrutura na qual o software é desenvolvido e implantado). Já o DevSecOps, é uma combinação das primeiras letras de “desenvolvimento”, “segurança” e “operações”.

A esse respeito, analise os itens a seguir.

I. DevSecOps é uma extensão da mudança cultural em direção ao DevOps, incorporando todos dentro da organização, incluindo partes interessadas nos negócios, segurança, arquitetos, proprietários de produtos e engenheiros de desenvolvimento e de operações.
II. DevOps é a colaboração de pessoal e processos de desenvolvimento e operações para formar uma única equipe de entrega ágil composta por partes interessadas, clientes, engenheiros e testadores. Essa equipe trabalha em conjunto para fornecer software e infraestrutura em ritmo acelerado por meio de um pipeline automatizado de CI/CD.

III. É natural supor que o DevSecOps una engenheiros de requisitos, de dados e de redes com DevOps em uma única equipe. No entanto, a segurança é criada por um grupo multidisciplinar de indivíduos externos à organização, cada um com uma função específica, testar falhas e explorar vulnerabilidades em sistemas e bancos de dados.

Está correto o que se afirma em

a) I, apenas.
b) III, apenas.
c) I, II e III.
d) I e II, apenas.
e) II e III, apenas.

06. (FGV/DNIT/2024) Infraestrutura como código (IaC) é uma forma de gerenciamento e provisionamento de infraestrutura por meio de código em vez de processos manuais.

Com relação às três práticas para implementar IaC, analise as afirmativas a seguir.

I. Definir todas as suas coisas como código é uma prática básica para fazer alterações rápidas e de modo confiável. Os principais motivos para a adoção dessa prática são a interoperabilidade, a consistência e a acessibilidade.
II. Testar e entregar continuamente todo o trabalho em andamento é uma prática onde as equipes de infraestrutura são rigorosas nos testes. Eles usam a automação para implantar e testar o sistema como um todo antes de testar e integrar todos os módulos desenvolvidos por todos os envolvidos.
III. Construir peças pequenas e simples, que os membros da equipe podem alterar independentemente é uma prática essencial, pois muitas equipes enfrentam dificuldades quando seus sistemas são grandes e fortemente acoplados. Quanto maior um sistema, mais difícil mudá-lo e mais fácil quebrar.

Está correto o que se afirma em

a) I e II, apenas.
b) I, apenas.
c) II, apenas.
d) III, apenas.
e) II e III, apenas.

07. (CEBRASPE/INPI/2024) A respeito de metodologias e técnicas prescritas pela engenharia de *software* para o desenvolvimento e para a gestão de produtos, julgue o item que se segue.

Na entrega de *software* funcionando para a produção de forma ágil, o DevOps prestigia a entrega contínua; nesse sentido, um princípio recomendado é que as atualizações de arquivo do servidor, por meio de uma operação de atualização de arquivos (commit) realizada pelos desenvolvedores, sejam centralizadas em uma máquina de integração.

08. (CEBRASPE/APEX/2024) Em uma arquitetura DevOps, existem diferenças entre a integração contínua e o *deployment* contínuo. Assinale a opção em que é apresentada característica aplicada ao *deployment* contínuo.

a) Torna novas entregas um não evento (não existindo mais um “dia D”), evitando, assim, que a perda de um *deadline* atrase a entrega de uma funcionalidade por meses.
b) *Builds* e testes devem ser executados com frequência, se possível após cada *commit* realizado no *master*.
c) É recomendável para sistemas *desktop*, *app* móvel ou embarcado que precisem de atualização.
d) Aumenta o tempo de entrega de novas funcionalidades, que são liberadas assim que ficam prontas, diminuindo o intervalo entre *releases* e o número de *releases*.

09. (CEBRASPE/MPO/2024) Acerca do DevOps, julgue o item seguinte.

Os recursos de monitoramento são de responsabilidade privativa, nessa abordagem, da equipe de operações no processo de CI/CD (continuous integration / continuous delivery).

10. (CEBRASPE/BACEN/2024) A respeito de infraestrutura como código, Docker e Kubernetes, julgue o item a seguir.

No modelo de infraestrutura como código, é possível a implantação de uma infraestrutura com definição de sub-redes, balanceadores de carga e máquinas virtuais com a utilização de código, sem a necessidade de configurações manuais.

11. (FGV/PGM NITERÓI/2023) Um Time de Desenvolvimento de Software (TDS) segue um protocolo automatizado para gerar, testar e combinar pacotes de software gerados separadamente. Todo software combinado precisa passar por um processo que inclui uma requisição formal ao Time de Operações (TO) de um Centro de Dados para executar um conjunto de testes, com o intuito de verificar vulnerabilidades no software antes de entrar em produção.

Considerando os conceitos de DevOps e DevSecOps, o TDS e o TO estão falhando no princípio:

a) automação;
b) colaboração ágil;
c) produção contínua;
d) integração contínua;
e) ação centrada no cliente.

12. (CEBRASPE/SERPRO/2023) Acerca da gestão de configuração, julgue o item a seguir.

A prática de database migration é fundamental em DevOps para garantir a consistência dos dados bem como sua disponibilidade e segurança.

13. (CEBRASPE/EMPREL/2023) No DevOps, existe uma prática em que novas alterações de código de uma solução de software são criadas, testadas e mescladas regularmente em um repositório central, com objetivo de encontrar e investigar bugs mais rapidamente. Trata-se da

a) integração contínua.
b) entrega contínua.
c) implantação em contêineres.

d) disponibilização em microsserviços.
e) pipeline.

14. (CEBRASPE/PREF. FORTALEZA/2023) Tendo em vista que, no atual cenário de desenvolvimento de aplicações web, é essencial considerar princípios, como consistência e escalabilidade, e práticas, como automação do processo de implantação e integração do código-fonte, julgue o item subsequente.

As ferramentas DevOps incluem o Kubernetes, uma plataforma de orquestração de contêineres que permite gerenciar e escalonar aplicativos em contêineres em diferentes ambientes e pode ser integrada a outras ferramentas DevOps, como Docker e Jenkins.

15. (CEBRASPE/PREF. FORTALEZA/2023) Julgue o item que se segue, com relação a técnicas de integração e implantação contínua de código.

Embora as técnicas de integração contínua sejam essenciais para garantir a qualidade do software e a eficiência do processo de desenvolvimento, elas não possibilitam a realização de testes automatizados de aceitação.

16. (CEBRASPE/MPE RO/2023) Determinada empresa adota a cultura DevOps, com os objetivos de colaboração e integração do desenvolvimento de software e das operações. A mesma empresa também adota práticas de automação em que alterações de código são frequentes e adicionadas a um repositório central de versionamento, para que, em seguida, ocorram as etapas de build e teste para melhorar a qualidade.

Nessa situação hipotética, a empresa aplica o conceito de

a) agile scrum.
b) continuous integration.
c) machine learning.
d) blockchain.
e) continuous delivery.

17. (CEBRASPE/DATAPREV/2023) A respeito de DevSecOps e de técnicas de análise de desempenho, julgue o item a seguir.

O DevSecOps deve evitar que o ciclo de trabalho do DevOps fique muito lento, observando sempre a segurança da aplicação e a infraestrutura, e, para isso, um dos cuidados que se deve ter é evitar automatizar barreiras de segurança, o que pode inserir pontos de fragilidade no software, comprometendo a segurança e o tempo de desenvolvimento.

18. (CEBRASPE/DATAPREV/2023) A respeito de DevSecOps e de técnicas de análise de desempenho, julgue o item a seguir.

O DevSecOps trata de segurança integrada, e não da segurança que funciona no perímetro em torno de aplicativos e dados, a qual também inclui a identificação dos riscos que estão envolvidos na cadeia de fornecimento do software.

19. (CEBRASPE/DATAPREV/2023) Tendo como referência conceitos e princípios de DevOps e DevSecOps, julgue o item a seguir, relativo aos 6 Cs.

O processo de entrega contínua envolve a automação de processos de desenvolvimento, testes e integração dos códigos de forma frequente e em um repositório compartilhado.

20. (CEBRASPE/PETROBRAS/2022) Julgue o item subsecutivo, relativo a DevOps e notação BPMN.

No DevOps, a integração contínua possui como uma de suas atividades a realização de testes; a fim de se obter os benefícios esperados convém automatizar os testes para poder executá-los para cada alteração feita no repositório principal.

21. (FCC/PGE AM/2022) A transição de DevOps para DevSecOps requer a compreensão e utilização de técnicas e práticas específicas que podem garantir a segurança do software. Uma especialista em Engenharia de Software recomendou, dentre outras, as seguintes ferramentas e tecnologias para essa transição em uma empresa:

> I. É usado para verificar o código sem realmente executá-lo. Este tipo de ferramenta ajuda a encontrar vulnerabilidades em potencial no código-fonte, evitando que ocorram várias vulnerabilidades do tipo zero-day. Common Weakness Enumeration (CWE) é uma das classificações de avisos mais comuns produzidos por estas ferramentas. CWE é uma lista oficial ou dicionário de pontos fracos de segurança comuns exploráveis por invasores para obter acesso não autorizado ao sistema.
> II. Da mesma forma que as ferramentas que executam testes de caixa preta, estes analisadores dinâmicos podem identificar vulnerabilidades do programa, como injeções de SQL, estouros de buffer e similares.
> III. Este tipo de ferramenta analisa o comportamento do aplicativo, implementando uma análise de segurança contínua, sendo uma das tecnologias de segurança usadas em tempo de execução.

Os itens I, II e III correspondem, correta e respectivamente, a

a) RASP (Runtime Application Self-Protection) − SAST (Static Application Security Testing) − IAST (Interactive Application Security Testing).
b) IAST (Interactive Application Security Testing) − SAST (Static Application Security Testing) − DAST (Dynamic Application Security Testing).
c) SAST (Static Application Security Testing) − IAST (Interactive Application Security Testing) − DAST (Dynamic Application Security Testing).
d) IAST (Interactive Application Security Testing) − SAST (Static Application Security Testing) − RASP (Runtime Application Self-Protection).
e) SAST (Static Application Security Testing) − DAST (Dynamic Application Security Testing) − RASP (Runtime Application Self-Protection).

22. (FCC/TRT 4/2022) Integração e implantação contínuas (CI/CD) é uma coleção de práticas usadas para integrar e entregar código rapidamente. Usando a prática de

a) integração contínua (CI), em um cronograma pré-acordado, provavelmente no final do ciclo de sprint, o pacote de código é liberado e testado manualmente pelo time de desenvolvimento.
b) implantação contínua (CD), depois que todo o código é validado com sucesso pelo time de desenvolvimento e qualidade, implanta-se o código manualmente no ambiente de teste ou de produção.
c) implantação contínua (CD) em que o grupo controle de qualidade realiza testes manuais para todos os códigos novos ou modificados. Se for bem-sucedido, o código será liberado para implantação; caso contrário, o lançamento é rejeitado e colocado em espera até que os bugs sejam corrigidos.
d) integração contínua (CI), os desenvolvedores são incentivados a fazer pequenos ajustes de código, mesclar seu código na ramificação de código principal e validá-lo usando testes automatizados.
e) integração contínua (CI), para implementar pipelines devem ser utilizadas ferramentas como Zuul e Eureka.

23. (FGV/TRT 16/2022) Considerando o DevOps e suas boas práticas, analise os itens a seguir:

I. Testes integrados são uma parte importante do processo DevOps. Esses testes devem levar em consideração as práticas de Test-Driven Development e Behavior-Driven Development, dessa forma a execução automática desses testes pode ser integrada ao pipeline de CI. No entanto, é importante integrar outros tipos de testes, como testes funcionais ou testes de integração, que permitem que o aplicativo seja testado funcionalmente do início ao fim com os outros componentes do seu ecossistema.
II. Recomenda-se automatizar apenas as tarefas críticas que envolvam poucas atualizações na implementação e nos testes dos aplicativos nas infraestruturas. Essas tarefas devem ser automatizadas em scripts que podem ser facilmente integradas e executadas em pipelines de CI/CD.
III. A construção de pipelines de CI/CD envolvem a escolha de ferramentas de DevOps adequadas pelas equipes considerando a natureza da empresa. É necessário levar em conta aspectos financeiros, avaliar entre ferramentas de código aberto e gratuitas e as proprietárias, que são mais ricas em recursos e suporte, mas exigem um investimento significativo.

Está correto apenas o que se afirma em

a) I e III.
b) I.
c) I e II.
d) II e III.
e) II.

24. (CEBRASPE/ALECE/2021) DevOps, termo criado para descrever um conjunto de práticas para integração entre as equipes de desenvolvimento de softwares, tem como princípio

a) utilizar padrões de projeto para problemas recorrentes e conhecidos no desenvolvimento de software.
b) implantar a solução de software por meio de processos repetíveis e confiáveis.
c) desenvolver o software por meio funcionalidades implementadas e disponibilizadas na forma de serviços.
d) desenvolver soluções em no mínimo três camadas: de interação (usuário), de manipulação dos dados (modelo) e de controle.
e) utilizar serviço de computação baseado na Internet por meio de recursos de TI sob demanda.

25. (FCC/TRF 4/2019) Um Analista de TI, ao pesquisar sobre testes automatizados, encontrou a seguinte informação:

Para escrever os testes de aceitação, podemos usar os critérios de aceitação descritos nas histórias de usuário. Estando estes testes implementados, executando e passando com sucesso, obter-se-á uma boa cobertura e garantia de que pelo menos as funcionalidades mais importantes estão sendo de fato cobertas pelos testes. É recomendável que esses testes sejam executados por um tipo de ferramenta (como: Jenkins, TravisCI, Hudson, CruiseControl etc.), que fará o trabalho de executar os testes automatizados de acordo com alguma estratégia predefinida (como: todo dia, a cada modificação feita no código-fonte etc.).

O Analista conclui que o tipo de ferramenta mencionada no texto é de

a) teste unitário.
b) integração contínua.
c) teste de usabilidade.
d) teste cross-platform.
e) DDT- Database Driven Testing.

02648216103 - HENRIQUE AMORIM BARRETO

GABARITO

GABARITO



1. Letra C
2. Certo
3. Errado
4. Certo
5. Letra D
6. Letra D
7. Certo
8. Letra A
9. Errado
10. Certo
11. Letra A
12. Certo
13. Letra A
14. Certo
15. Errado
16. Letra B
17. Errado
18. Certo
19. Errado
20. Certo
21. Letra E
22. Letra D
23. Letra A
24. Letra B
25. Letra B

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. 

Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. 

Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. 

Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. 

Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. 

Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. 

Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. 

Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. 

O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.